

Gamification Tool Development: An Ontology-Driven and AI-Assisted Lifecycle Framework

Furkan Efe Zerenuz
Dept. of Computer Engineering
Istanbul Kultur University
2200004014

Zeynep Tokusoglu
Dept. of Computer Engineering
Istanbul Kultur University
2200005823

Furkan Gencer
Dept. of Computer Engineering
Istanbul Kultur University
2200004309

Abstract—This work introduces a flexible gamification platform built to turn passive learning and workplace settings into dynamic interactions [1]. Built across layers with React [10], FastAPI [9], and PostgreSQL, its core lies in a structured approach shaped by the 6D method plus twenty-five specific design rules [2]. Rather than follow standard tools, it includes artificial intelligence that offers instant feedback on structure, checks goal coherence, while verifying relationships through OWL/SPARQL logic [7]. What emerges is a system where incentives, game elements, and aims stay aligned without manual oversight. Starting with design files, the Realization Engine builds deployment packages automatically. These outputs feed into a 2D Runtime powered by Phaser, bringing simulated labs to life through interaction.

Index Terms—Gamification Engine, 6D Framework, AI-Assisted Design, OWL/SPARQL, Software Architecture.

I. INTRODUCTION

The aim is to build an end-to-end ontology-driven gamification platform [6] that supports the complete gamification lifecycle from specification to realization. Current gamification efforts often rely on loosely organized documentation, which complicates the verification of logical flow and consistency. To address this, we propose a platform with two integrated capabilities: a specification generator that iteratively builds comprehensive 25-section gamification design documents based on user requirements, and a realization engine that transforms approved specifications into deployable gamification solutions.

The tool accepts gamification requirements, iteratively builds detailed specification documents through stakeholder review cycles, and once a specification is approved, generates the necessary components to realize the gamification. This bridges the gap between gamification theory and practical implementation. The platform utilizes GamifyOnt, a specialized ontology backed by OWL and SPARQL [7], ensuring that every point, level, and player interaction remains semantically aligned. An AI helper offers real-time advice and consistency reviews during drafting.

II. RELATED WORK

Though often explored, turning tasks into game-like experiences mainly aims to boost drive, involvement, or how well people learn. Research tends to center around building systems that include gameplay features, picking mechanics that fit certain settings, or adjusting those setups for particular users. Ontology-driven gamification setups encode gamification concepts using structured formats.

Systems like GaTO, AGE-Learn, and OntoGamif demonstrate modeling of players, objectives, game mechanics, layer and motivational drivers through formal frameworks. While they excel at capturing meaning precisely, their application frequently stops at testing ideas or serving narrow fields. Model-based gamification methods convert design structures into working components. However, such strategies tend to skip AI support for writing specs or adaptable ways to bring documents in and out. Rarely do they cover everything needed: defining needs, checking meaning through logic models, automated guidance during development, producing elements while running, and tracking user behavior dynamically.

Table 1

LIFECYCLE COVERAGE COMPARISON

Lifecycle Stage	Frameworks	Ontology Sys.	AI Tools	Proposed
Requirement Collection	Partial	Partial	Limited	Yes
Specification Mgmt.	Limited	Limited	No	Yes
Semantic Validation	No	Yes	No	Yes
AI Improvement	No	Limited	Yes	Yes
Runtime Realization	No	No	No	Yes
Gameplay Execution	No	Limited	No	Yes
Analytics & Telemetry	Limited	Limited	No	Yes

As shown in Table I, our proposed system integrates ontology reasoning, AI-assisted development, and runtime realization within a single cohesive architecture.

III. METHODOLOGY

The methodology follows a modular software engineering approach, dividing the system into requirement analysis, semantic validation [5], and runtime execution layers [8]. Built around heavy software integration, the solution combines a frontend using React with a FastAPI-powered backend, tied to a PostgreSQL data store. Alongside these components run semantic backend, tied to a PostgreSQL data store. Alongside these components run semantic validation tools guided by ontologies, design aids enhanced through artificial intelligence, while a dedicated gamification engine manages live interactions.

A. Requirement Analysis and Workflow

The platform serves three distinct roles: Creators (Designers), Managers (Administrators), and Participants (Players). The core functionality revolves around the 25-section specification template, which ensures comprehensive coverage of game mechanics. A set of distinct user roles shapes how people engage with the platform. The system initializes the 25-section template, and the user fills specification sections manually or with AI assistance. The overall activity flow, from authentication to runtime access, is detailed in Fig. 1.

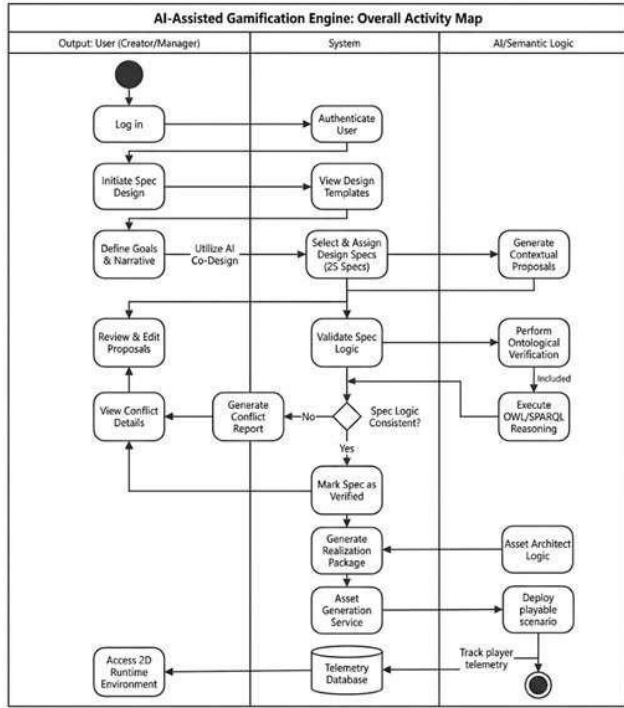


Figure 1 Activity Diagram

A. Semantic Logic and Validation

Validation is handled via Owlready2, OWL, rdflib, and SPARQL. The user submits the specification for validation, and the ontology validation module checks semantic consistency using OWL/SPARQL rules. The system returns warnings or validation errors, allowing the user to revise the specification if necessary.

B. Implementation and Deployment

The backend logic is powered by Python via FastAPI and Uvicorn, while data persistence is managed by PostgreSQL and Alembic migrations. Once the user approves the validated specification, the realization engine generates runtime configuration files. Runtime assets, mechanics, rules, and progression structures are created. These outputs feed into a 2D Runtime powered by Phaser, bringing simulated labs to life through interaction.

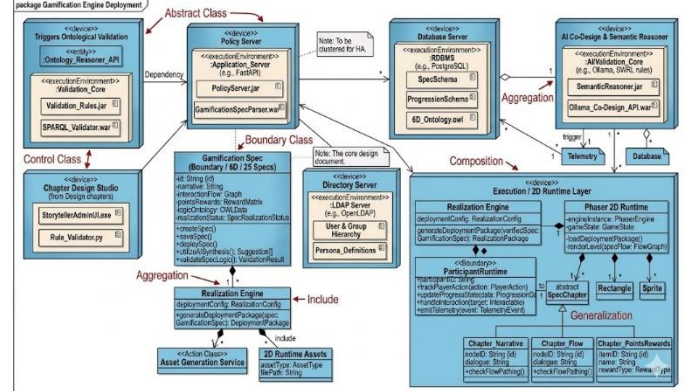


Figure 2 Deployment Diagram

As illustrated in the deployment diagram (Fig. 2), the in-structure supports distributed execution across servers[8].

IV. EXPERIMENTAL RESULTS AND DISCUSSION

System performance was evaluated based on the speed of AI-assisted design and the error-catching capability of the ontological validation. Because the ontology layer checked links among mechanics, rewards, KPIs, telemetry needs, and progression systems, meaning stayed consistent. Meanwhile, generating content with AI sped up how fast specs were built.

Table 2

Summary of Ontology Validation Performance

Test Variant	Status	Wall Time (ms)	Observation
Cold Start (empty_25)	Success	22.745	Initial OWL loading overhead
Warm Cache (valid_rich)	Success	0.027	High-speed iterative validation
Constraint Violation	Failed	< 0.020	Immediate logic error detection

Despite the initial delay shown in Table 2—about 22.7 ms

needed to start the OWL reasoner—the system quickly shifts into faster operation. After first use, cached data takes over, and validation then runs much faster, dropping to just 0.027 ms. Because of this shift from cold to warm cache behavior, reasoning tasks stay out of the way while coding progresses.

Table 3

Realization Performance Metrics

Scenario	JSON Size (KB)	Execution Time (ms)	Package Rules
Initial Load (Empty_25)	0.999	2206.59	5
Optimized Load (Valid Rich)	7.508	912.45	3

REFERENCES

- [1] J. Hamari, J. Koivisto, and H. Sarsa, “Does Gamification Work? — A Literature Review of Empirical Studies on Gamification,” in *Proceedings of the 47th Hawaii International Conference on System Sciences*, 2014, pp. 3025–3034.
- [2] B. Morschheuser, K. Werder, J. Hamari, and J. Abe, “How to Design Gamification? A Method for Engineering Gamified Software,” *Information and Software Technology*, vol. 95, pp. 219–237, 2018.
- [3] S. Izumi, F. O. Okeyo, and K. Matsuo, “Ontology-Based Gamification Framework for Intelligent Learning Environments,” *Frontiers in Artificial Intelligence*, vol. 2, 2019.
- [4] A. Calderón, M. Ruiz, and M. O’Connor, “GDF: A Gamification Design Framework Powered by Model-Driven Engineering,” *IEEE Access*, vol. 7, pp. 73037–73054, 2019.
- [5] Y. Li and P. Ralph, “Using Generative AI to Support Game Design and Development,” *Computers*, vol. 13, no. 6, 2024.
- [6] T. Gruber, “A Translation Approach to Portable Ontology Specifications,” *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.
- [7] I. Horrocks, P. F. Patel-Schneider, and F. van Harmelen, “From SHIQ and RDF to OWL: The Making of a Web Ontology Language,” *Journal of Web Semantics*, vol. 1, no. 1, pp. 7–26, 2003.
- [8] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O’Reilly Media, 2020.
- [9] FastAPI Documentation, “FastAPI Framework,” 2025.
- [10] React Documentation, “React,” 20

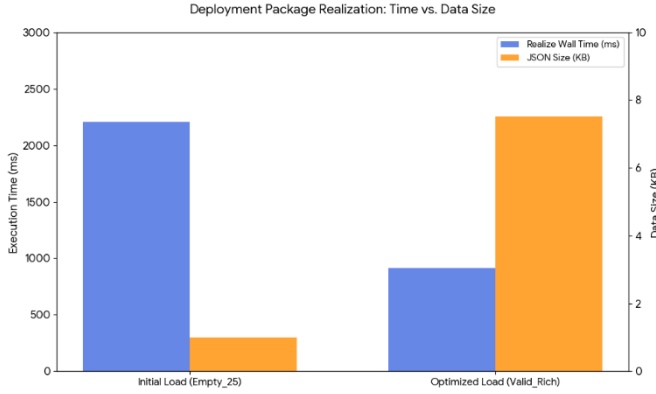


Figure 3: Comparison of Execution Time vs. Data Complexity

Figure 3 shows how bigger data led to faster performance in these tests. Though the starting run took about 2200 milliseconds, the next one handled a much larger JSON file - 7.5 KB - in just over 912 milliseconds. That speed gain came because the Java Virtual Machine had warmed up, along with improved readiness of the HermiT reasoner after first use. Once running, the system handles tough cases very well. Details on hardware needs appear in Appendix B.

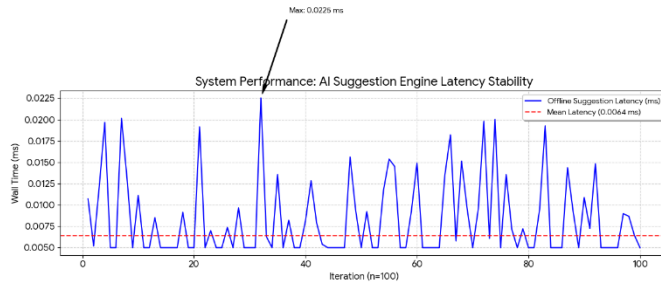


Figure 4 Latency Stability for 100 Consecutive AI Suggestions

Measured results show near-instant response times across trials. Stability stands clear when viewing Figure 4. Averaged delay hits just 0.0064 milliseconds - almost imperceptible. At its slowest point, system lag still falls short of detection thresholds tied to human reaction speed. This means design feedback arrives instantly, with no drop in operational smoothness.

V. CONCLUSION

This work tackled the absence of a cohesive platform for managing every phase of gamified systems from initial design through testing, deployment, implementation, and performance tracking. By blending React interaction, FastAPI logic, and OWL-based semantic validation, we have demonstrated a technical possibility that yields real-world usefulness. The project proves that gamification plans can shift from static papers into live, data-driven systems capable of reporting outcomes in real-time. Future research will focus on cloud-native scalable deployment architectures and multiplayer collaborative runtime systems.

ACKNOWLEDGMENT

Special thanks to Asst. Prof. Dr. Yusuf Altunel for his steady direction and consistent engagement throughout this project. We also acknowledge the Istanbul Kultur University Department of Computer Engineering for providing the necessary tools and scholarly space.

